TÜBİTAK 2209-A — Üniversite Öğrencileri Araştırma Projeleri Destek Programı

Alg Üretim Havuzları için Çok Parametreli Su Kalitesi İzleme ve Uzaktan Yönetim Sistemi

Proje Sonuç Raporu

Proje Yürütücüsü	
Danışman	
Kurum / Bölüm	
Tarih	Haziran 2026

İçindekiler

1. Özet
2. Giriş
3. Amaç ve Kapsam
4. Sistem Mimarisi
5. Donanım Tasarımı ve Ölçüm Parametreleri
6. Gömülü Yazılım (Firmware)
7. Veri Akışı ve Haberleşme Protokolü
8. Veritabanı Tasarımı
9. Güvenlik ve Yetkilendirme
10. Alarm Modeli ve Veri Güncelliği
11. Web Yönetim Paneli
12. Doğrulama ve Bulgular
13. Sonuç ve Değerlendirme
14. Kaynakça
15. Ek A — Gömülü Yazılım Kaynak Kodu
16. Ek B — Veritabanı Şeması (SQL)

1. Özet

Bu çalışmada, mikroalg üretiminde verim ve süreç kararlılığı açısından kritik olan su kalitesi ve çevresel koşulların sürekli izlenmesi amacıyla, düşük maliyetli ve uzaktan yönetilebilen bir gömülü izleme sistemi geliştirilmiştir. Sistem; pH, elektriksel iletkenlik (EC), toplam çözünmüş katı (TDS), çözünmüş oksijen (DO), ışık şiddeti, su sıcaklığı, hava sıcaklığı ve bağıl nem olmak üzere sekiz parametreyi periyodik olarak ölçen bir ESP8266 tabanlı kenar düğümünden, ölçümleri merkezî olarak saklayan Supabase/PostgreSQL veri katmanından ve kullanıcıya canlı izleme ile yapılandırma imkânı sunan React tabanlı bir web panelinden oluşur. Analog problar 16-bit ADS1115 çevirici üzerinden, sayısal sensörler ise mikrodenetleyicinin doğrudan giriş pinleri üzerinden okunur. Tanımlı eşiklerin aşılması durumunda hem yerel sesli/görsel uyarı üretilir hem de olay kaydı merkezî veritabanına işlenir. Eşik ve alarm yapılandırması web panelinden değiştirilebilir; cihaz bu yapılandırmayı periyodik olarak çeker ve enerji kesintilerine karşı kalıcı bellekte önbelleğe alır. Geliştirilen mimari; ölçüm toplama, veri saklama, kullanıcıya bağlı cihaz yönetimi, eşik tabanlı alarm üretimi ve raporlanabilir veri üretimi işlevlerini bütünleşik bir prototipte bir araya getirmektedir.

2. Giriş

Mikroalg kültürleri; biyoyakıt, gıda takviyesi, yem ve atık su arıtımı gibi pek çok alanda kullanım potansiyeline sahip olup üretim verimliliği büyük ölçüde kültür ortamının fiziksel ve kimyasal koşullarına bağlıdır. pH, sıcaklık, çözünmüş oksijen, iletkenlik ve ışık gibi değişkenlerdeki dengesizlikler büyüme hızını düşürmekte, hatta kültürün çökmesine yol açabilmektedir. Bu nedenle havuz koşullarının düzenli olarak ölçülmesi ve sınır değerlerin aşıldığı durumlarda hızlı müdahale edilmesi önem taşır.

Geleneksel manuel ölçüm yaklaşımları zaman alıcıdır, kesintili veri üretir ve hızlı değişen koşulları yakalamakta yetersiz kalır. Düşük maliyetli mikrodenetleyiciler, modern sensör modülleri ve bulut tabanlı veri servislerinin yaygınlaşması, bu izleme sürecinin otomatikleştirilmesini ve uzaktan yönetilebilir hale getirilmesini mümkün kılmaktadır. Bu çalışmada söz konusu olanaklardan yararlanılarak; ölçümü, veri kaydını, uzaktan yapılandırmayı ve alarm üretimini tek bir uçtan uca sistemde birleştiren bir prototip tasarlanmış ve gerçeklenmiştir.

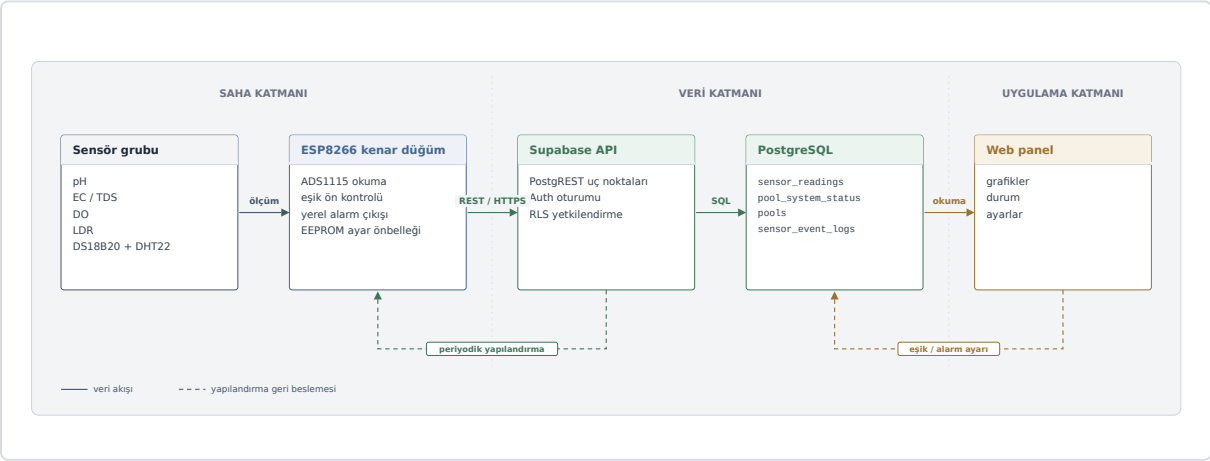
3. Amaç ve Kapsam

Projenin amacı, alg üretim havuzlarındaki kritik su kalitesi ve çevresel değişkenleri düzenli aralıklarla ölçen, ölçüm verisini merkezî bir veritabanında saklayan, kullanıcıya canlı bir panel üzerinden sunan ve eşik ihlallerinde alarm/olay kaydı üreten bir izleme sisteminin prototipini oluşturmaktır. Kapsam dahilinde aşağıdaki işlevler hedeflenmiştir:

- Sekiz parametrenin (pH, EC, TDS, DO, ışık, su sıcaklığı, hava sıcaklığı, nem) periyodik ölçümü ve fiziksel birime dönüştürülmesi,
- Ölçümlerin güvenli bir bağlantı üzerinden merkezî veritabanına aktarılması ve zaman serisi olarak saklanması,
- Kullanıcı hesabına bağlı, çok havuzlu cihaz yönetimi ve satır düzeyinde erişim denetimi,
- Web panelinden eşik/alarm yapılandırması ve bu yapılandırmanın cihaza yansıtılması,
- Eşik dışı durumlarda yerel uyarı (sesli/görsel) ve merkezî olay günlüğü üretimi,
- Bağlantı kesintilerine karşı dayanıklılık ve yapılandırmanın kalıcı bellekte önbelleğe alınması.

4. Sistem Mimarisi

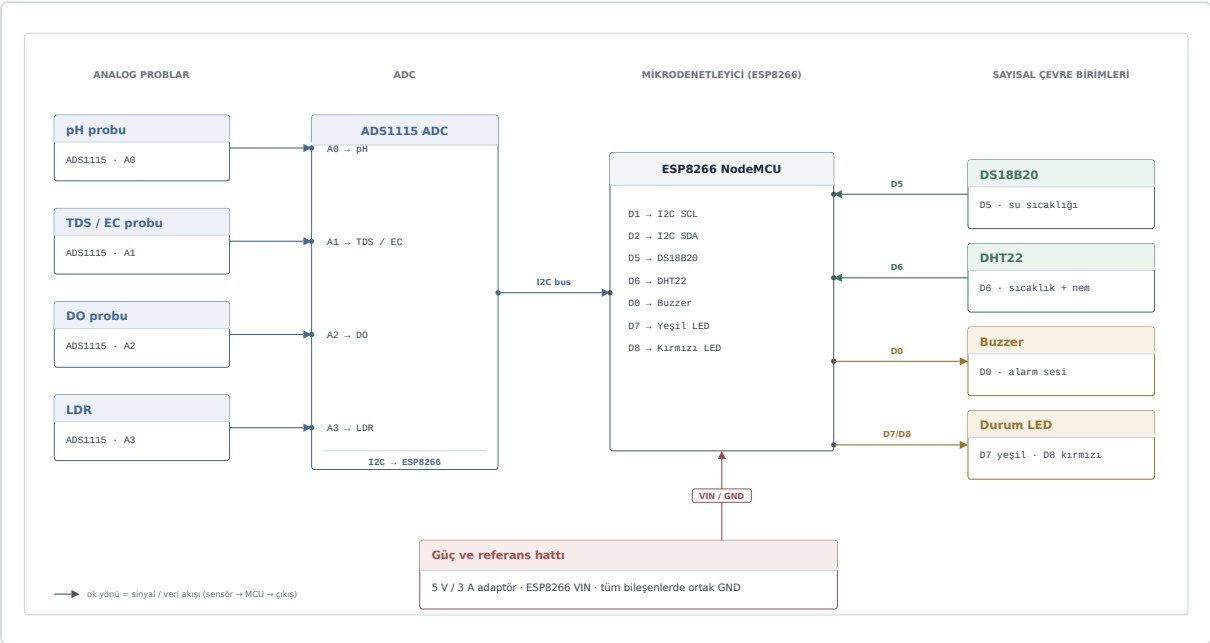
Sistem; saha ölçüm katmanı, kenar düğümü, veri katmanı ve uygulama katmanından oluşan katmanlı bir mimariye sahiptir (Şekil 1). Saha katmanında analog ve sayısal sensörler bulunur. Kenar düğümü olarak görev yapan ESP8266, sensör verilerini okur, fiziksel birimlere dönüştürür, eşik ön kontrolünü yapar ve sonuçları HTTPS üzerinden Supabase API'sine iletir. Veri katmanında Supabase tarafından sağlanan PostgREST uç noktaları, kimlik doğrulama ve satır düzeyi güvenlik (RLS) politikalarıyla korunan PostgreSQL veritabanı yer alır. Uygulama katmanındaki web paneli, ölçüm ve durum verisini okuyarak grafikler ve kartlar halinde sunar; eşik/alarm ayarlarını ise veritabanına geri yazarak cihazın davranışını uzaktan etkiler.



Şekil 1. Sistemin katmanlı genel mimarisi: saha sensörlerinden web paneline ölçüm akışı ve yapılandırma geri beslemesi.

5. Donanım Tasarımı ve Ölçüm Parametreleri

Kenar düğümü, dahili Wi-Fi modülü ve yeterli giriş/çıkış kapasitesi nedeniyle ESP8266 NodeMCU kartı üzerine kurulmuştur. Mikrodenetleyicinin tek bir analog girişi bulunduğundan, dört analog prob (pH, TDS/EC, DO ve ışık-LDR) 16-bit çözünürlüklü ADS1115 analog-sayısal çeviriciye bağlanmış ve çevirici mikrodenetleyiciye I²C arabirimi (SCL→D1, SDA→D2) üzerinden bağlanmıştır. Su sıcaklığı için OneWire protokollü DS18B20, hava sıcaklığı ve bağıl nem için DHT22 sensörü mikrodenetleyicinin sayısal pinlerine (sırasıyla D5 ve D6) doğrudan bağlanır. Eşik dışı durumların yerel olarak bildirilmesi için bir buzzer (D0) ve bağlantı durumunu gösteren iki durum LED'i (D7 yeşil, D8 kırmızı) kullanılır. Tüm bileşenler 5 V / 3 A adaptör ile beslenir ve ortak toprak hattı paylaşır.



Şekil 2. Donanım bağlantı şeması ve pin eşleştirmesi. Ok yönleri sinyal/veri akışını gösterir: sensörler mikrodenetleyiciye girer, çıkış birimleri sürülür.

Ölçülen parametreler, her birine atanmış sensör kaynağı, fiziksel birim ve veritabanında kullanılan sensör kimliği Şekil 12'de özetlenmiştir. Analog problardan okunan gerilim değerleri, firmware içinde sensöre özgü dönüşüm bağıntılarıyla fiziksel birimlere çevrilir; örneğin sıcaklık telafili TDS hesabı ve ışık şiddetinin yüzdeye normalize edilmesi bu aşamada yapılır.

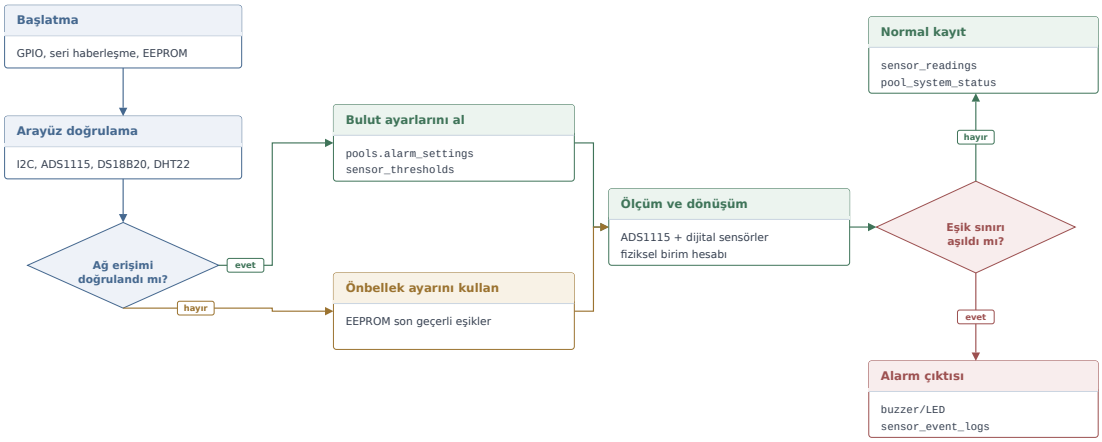
Parametre	Sensör / kaynak	Birim	Veri kimliği
pH	pH probu -> ADS1115 A0	-	ph
EC	TDS/EC probu -> ADS1115 A1	mS/cm	ec
TDS	TDS/EC probu -> ADS1115 A1	ppm	tds
Çözünmüş oksijen	DO probu -> ADS1115 A2	mg/L	do
Işık	LDR -> ADS1115 A3	%	ldr
Su sıcaklığı	DS18B20 -> D5	°C	waterTemp
Hava sıcaklığı	DHT22 -> D6	°C	airTemp
Nem	DHT22 -> D6	%	humidity

Şekil 12. Ölçüm parametreleri, sensör kaynakları, birimleri ve veritabanı sensör kimlikleri.

6. Gömülü Yazılım (Firmware)

Mikrodenetleyici yazılımı; başlatma, arabirim doğrulama, ağ erişimi, yapılandırma çekme, ölçüm/dönüşüm ve eşik değerlendirmesi adımlarından oluşan bir döngü ile çalışır (Şekil 5). Açılışta GPIO, seri haberleşme ve kalıcı bellek (EEPROM) ilklendirilir; ardından I²C üzerinden ADS1115, DS18B20 ve DHT22 arabirimleri doğrulanır. Ağ bağlantısı kurulabildiğinde cihaz, ait olduğu havuzun güncel eşik ve alarm yapılandırmasını veritabanından periyodik olarak çeker; bağlantı yoksa kalıcı bellekte saklanan son geçerli yapılandırma kullanılır. Bu önbellekleme, bir bütünlük imzası (magic, sürüm ve sağlama toplamı) ile korunur ve enerji kesintilerine karşı yapılandırmanın korunmasını sağlar.

Her ölçüm çevriminde analog ve sayısal sensörler okunur, fiziksel birimlere dönüştürülür ve her parametre tanımlı uyarı aralığıyla karşılaştırılır. Bir veya birden fazla parametre aralık dışına çıktığında bir "alarm anahtarı" üretilir; bu anahtar, hangi sensörün hangi yönde (düşük/yüksek) ihlal ettiğini kodlar. Alarm durumu değiştiğinde merkezî olay günlüğüne kayıt düşülür ve yerel buzzer, susturma kuralına uygun şekilde sürülür. Ölçüm periyodu, buzzer etkinliği ve sensör bazlı alarm etkinliği gibi ayarlar web panelinden değiştirilebilir ve cihaza yansıtılır.



Şekil 5. Firmware karar akışı: başlatma, ağ doğrulaması, yapılandırma kaynağının seçimi, ölçüm/dönüşüm ve eşik değerlendirmesi.

7. Veri Akışı ve Haberleşme Protokolü

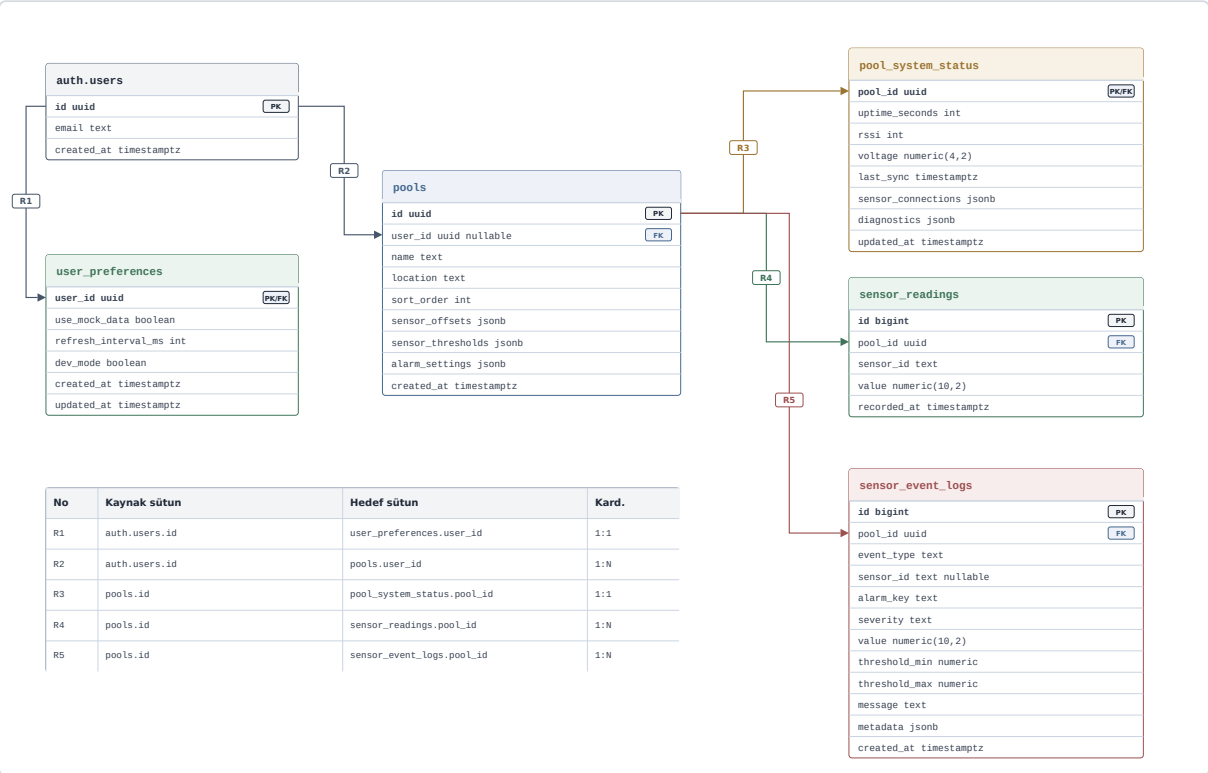
Cihaz ile veri katmanı arasındaki haberleşme, Supabase'in sağladığı PostgREST uç noktaları üzerinden JSON tabanlı HTTPS istekleriyle gerçekleştirilir. Kenar düğümü ölçüm paketlerini *sensor_readings* tablosuna ekler, cihaz sağlık bilgisini (sinyal gücü, besleme voltajı, tanı verileri, son senkron zamanı) *pool_system_status* tablosuna yazar ve güncel yapılandırmayı *pools* tablosundan okur. Web paneli ise kimlik doğrulamasının ardından ölçüm ve durum verisini okuyarak görselleştirir, eşik/alarm ayarlarını *pools* tablosundaki JSONB alanlara yazar ve alarm olaylarını *sensor_event_logs* tablosundan listeler. Bu okuma/yazma işlemlerinin kaynak, hedef ve kalıcı kayıt karşılıkları Şekil 3'te verilmiştir.

Adım	Kaynak	Hedef	İşlem	Kalıcı kayıt
1	ESP8266	sensor_readings	ölçüm paketi ekleme	pool_id, sensor_id, value, recorded_at
2	ESP8266	pool_system_status	cihaz sağlığı güncelleme	rssi, voltage, diagnostics, last_sync
3	ESP8266	pools	güncel yapılandırma okuma	sensor_thresholds, alarm_settings
4	Web panel	Supabase Auth	kimlik doğrulama	auth.users.id
5	Web panel	claim_pool()	cihazı kullanıcıya bağlama	pools.user_id
6	Web panel	readings + status	grafik ve kart verisi okuma	son ölçümler + durum
7	Web panel	pools	eşik/alarm ayarı kaydetme	JSONB yapılandırma
8	Web panel	sensor_event_logs	alarm olayı listeleme	event_type, severity, alarm_key

Şekil 3. Bileşenler arası veri akışı ve işlem protokolü; her adım için kaynak, hedef tablo ve kalıcı kayıt alanları.

8. Veritabanı Tasarımı

Veri modeli; kullanıcı tercihleri, havuz/cihaz kayıtları, cihaz sağlık durumu, zaman serisi ölçümler ve alarm olay günlükleri için ayrı tablolardan oluşur (Şekil 4). Kimlik yönetimi Supabase'in *auth.users* tablosu üzerinden sağlanır. *pools* tablosu havuzları temsil eder ve sensör kalibrasyon ofsetleri, eşikler ile alarm ayarlarını JSONB alanlarında tutar. Temel ilişkiler, *auth.users.id* alanından *user_preferences.user_id* ve *pools.user_id* alanlarına; *pools.id* alanından ise *pool_system_status.pool_id*, *sensor_readings.pool_id* ve *sensor_event_logs.pool_id* alanlarına kurulan birincil/yabancı anahtar bağlantılarıyla tanımlanmıştır. Ölçüm tablosunda sensör kimliği, geçerli parametre kümesini zorlayan bir CHECK kısıtı ile sınırlandırılmıştır.



No	Kaynak sütun	Hedef sütun	Kard.
R1	auth.users.id	user_preferences.user_id	1:1
R2	auth.users.id	pools.user_id	1:N
R3	pools.id	pool_system_status.pool_id	1:1
R4	pools.id	sensor_readings.pool_id	1:N
R5	pools.id	sensor_event_logs.pool_id	1:N

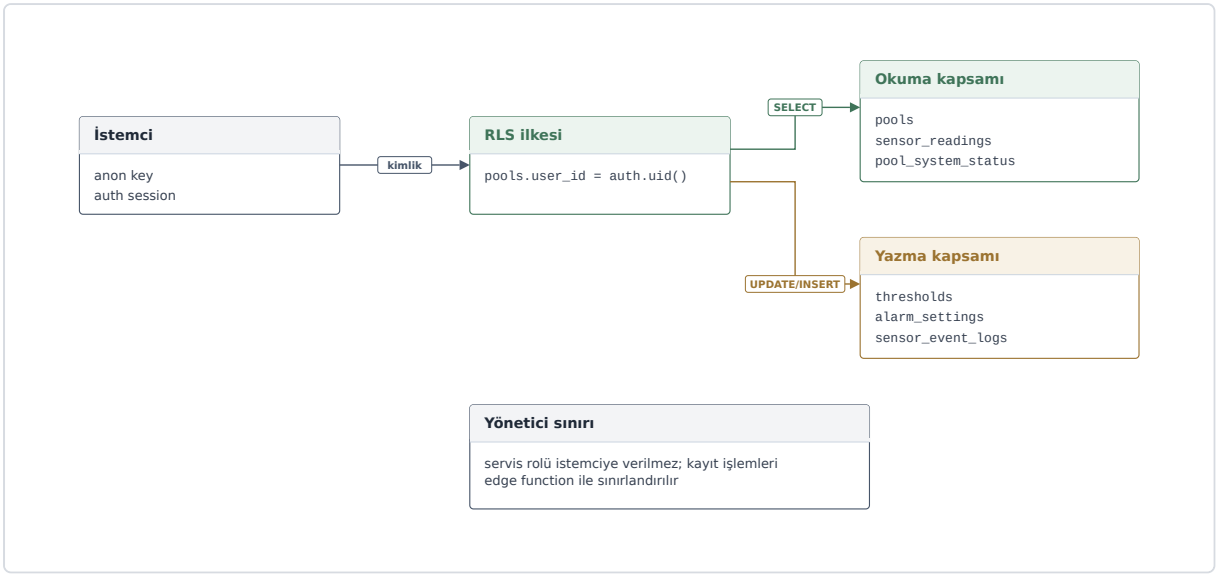
Şekil 4. Tam veritabanı ilişki diyagramı: tablolar, alanlar, birincil/yabancı anahtarlar ve kaynak-hedef ilişki eşleşmeleri.

9. Güvenlik ve Yetkilendirme

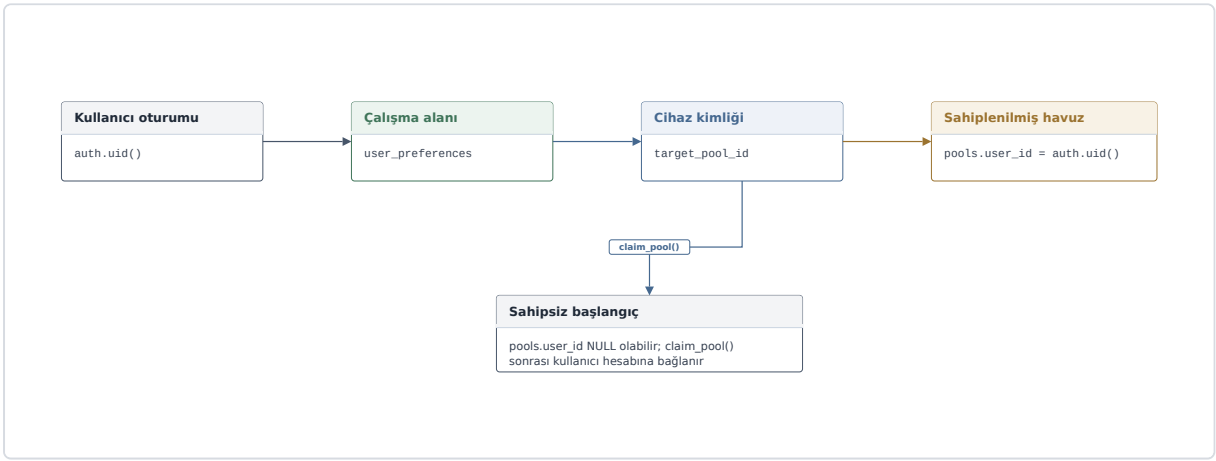
Web istemcisi, yalnızca anonim anahtar ve oturum jetonu ile veritabanına erişir; her tablo için tanımlanan satır düzeyi güvenlik (RLS) politikaları sayesinde bir kullanıcı yalnızca kendi havuzlarına ait satırları okuyabilir ve değiştirebilir. Politikaların özünü *pools.user_id = auth.uid()* eşitliği oluşturur (Şekil 10). Okuma kapsamı ölçüm ve durum tablolarını içerirken, yazma kapsamı eşik/alarm ayarları ve olay kayıtları ile sınırlıdır. Yüksek yetkili servis rolü istemciye verilmez.

Cihaz sahiplenme akışı, fabrikadan sabit bir havuz kimliğiyle gelen kenar düğümlerinin kullanıcı hesabına güvenli biçimde bağlanmasını sağlar (Şekil 8). Bir havuz başlangıçta sahipsiz olabilir

(*user_id* NULL); kullanıcı oturum açtıktan sonra *claim_pool()* işlemiyle cihaz, ilgili kullanıcı hesabına bağlanır ve bu noktadan itibaren RLS politikaları kapsamına girer.



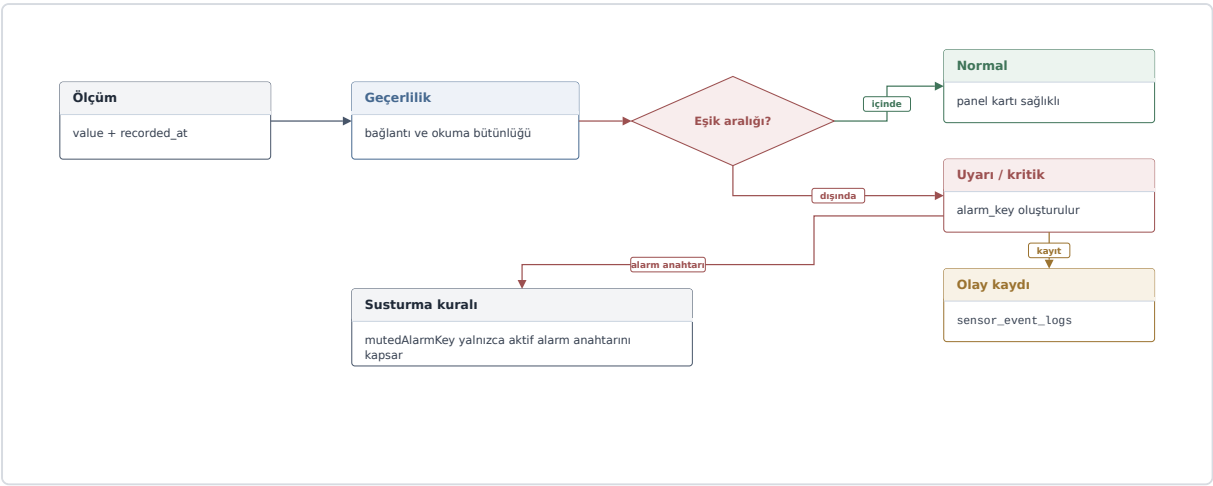
Şekil 10. Yetkilendirme ve veri erişim modeli: istemci kimliği, RLS politikası ile okuma/yazma kapsamlarının ayrımı.



Şekil 8. Cihaz sahiplenme akışı: oturum, çalışma alanı ve claim_pool() ile sahipsiz cihazın kullanıcı hesabına bağlanması.

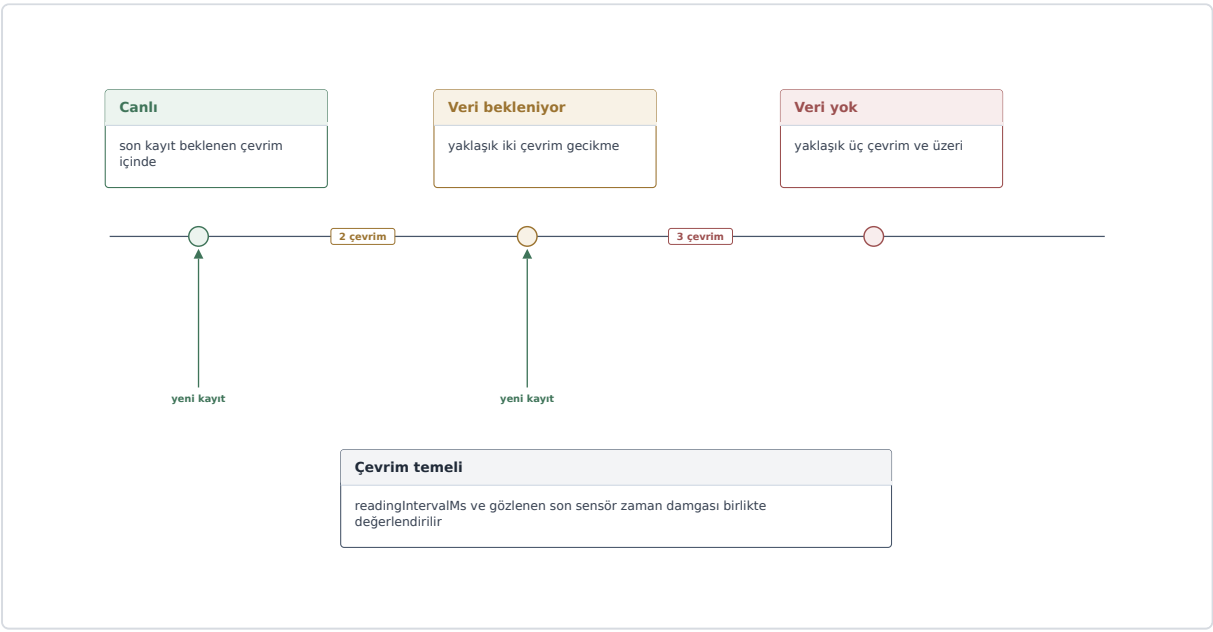
10. Alarm Modeli ve Veri Güncelliği

Alarm kararı, her ölçümün önce geçerlilik (bağlantı ve okuma bütünlüğü) açısından, ardından tanımlı eşik aralığı açısından değerlendirilmesiyle verilir (Şekil 6). Aralık içindeki değerler sağlıklı olarak işaretlenir; aralık dışındaki değerler için bir alarm anahtarı oluşturulur ve ilgili olay merkezî günlüğe kaydedilir. Susturma kuralı, yalnızca o an aktif olan alarm anahtarını kapsayacak biçimde tasarlanmıştır; böylece susturulan bir alarm, koşullar değiştiğinde otomatik olarak yeniden etkinleşir.



Şekil 6. Alarm karar modeli: ölçüm geçerliliği, eşik karşılaştırması, susturma kuralı ve olay günlüğü ilişkisi.

Panelde her havuzun veri güncelliği, cihazın bildirim çevrimine göre üç durumla ifade edilir (Şekil 7): son kayıt beklenen çevrim içindeyse "canlı", yaklaşık iki çevrim gecikme varsa "veri bekleniyor", yaklaşık üç çevrim ve üzeri gecikme varsa "veri yok". Bu değerlendirme, yapılandırılmış ölçüm periyodu ile gözlenen son ölçüm zaman damgasının birlikte ele alınmasıyla yapılır.

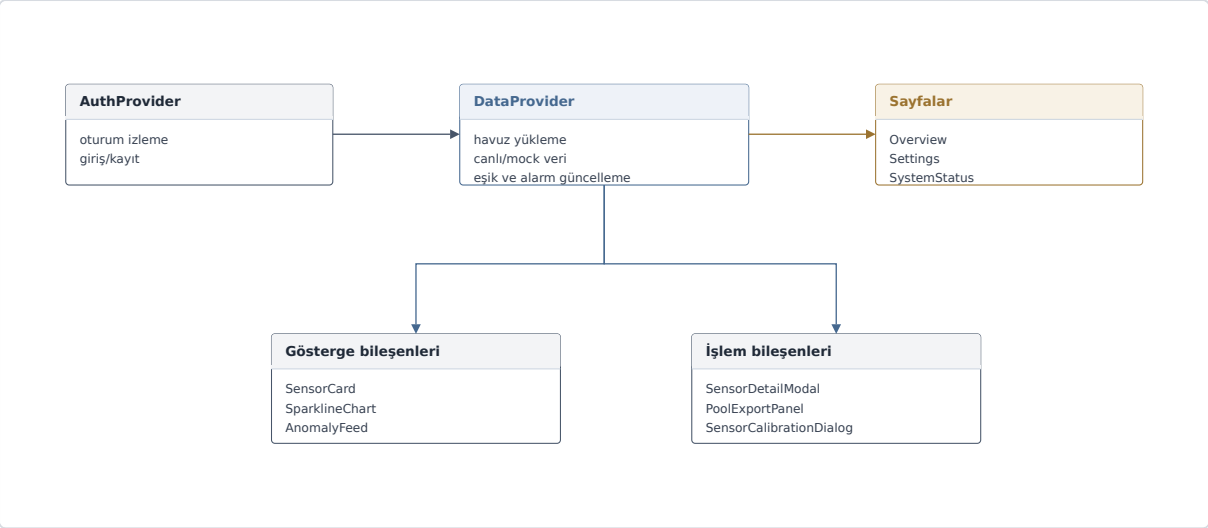


Şekil 7. Veri güncelliği durum modeli: canlı, bekleniyor ve veri yok durumlarının cihaz çevrimine göre ayrımı.

11. Web Yönetim Paneli

Web paneli, oturum yönetimini sağlayan bir kimlik sağlayıcısı, havuz verisini yükleyip canlı/örnek veri ile eşik-alarm güncellemelerini yöneten bir veri sağlayıcısı ve sayfalar (Genel Bakış, Ayarlar, Sistem Durumu) ile bunlara bağlı bileşenlerden oluşur (Şekil 9). Gösterge bileşenleri sensör kartları, ince çizgi grafikler ve anomali akışını; işlem bileşenleri ise sensör ayrıntı

penceresi, havuz dışa aktarımı ve kalibrasyon iletişim kutusunu içerir. Bu modüler yapı, görselleştirme ile kullanıcı işlemlerinin ayrıştırılmasını ve panelin sürdürülebilir biçimde genişletilmesini sağlar.



Şekil 9. Web panel modül mimarisi: kimlik/veri sağlayıcıları, sayfalar ve bileşenler arasındaki bağımlılıklar.

12. Doğrulama ve Bulgular

Sistemin işlevsel doğrulaması; donanım bağlantısı, ölçüm aktarımı, cihaz sağlığı, alarm davranışı ve panel kullanımı olmak üzere beş başlık altında yürütülmüştür (Şekil 11). Her başlık için beklenen çıktı, ilgili tablodaki kayıtlar ve panel görünümü üzerinden gözlemlenebilir kanıtlarla eşleştirilmiştir.

- **Donanım bağlantısı:** ADS1115, I²C ve GPIO arabirimlerinin başlatılması ve sensör bağlantı sürekliliği başarıyla gözlemlenmiştir; bağlantısı kopan sensörler durum bayraklarıyla işaretlenmektedir.
- **Ölçüm aktarımı:** Sekiz parametre zaman damgalı satırlar olarak *sensor_readings* tablosuna periyodik biçimde işlenmiştir.
- **Cihaz sağlığı:** Sinyal gücü, besleme voltajı ve son senkron zamanı *pool_system_status* tablosunda güncel tutulmuştur.
- **Alarm davranışı:** Eşik dışı değerlerde alarm anahtarı üretilmiş, olay türü ve önem derecesiyle birlikte *sensor_event_logs* tablosuna kaydedilmiştir.
- **Panel kullanımı:** Grafik, kart ve dışa aktarım işlevleri, okunabilir kullanıcı çıktıları üretmiştir.

Doğrulama alanı	Kanıt	Beklenen çıktı
Donanım bağlantısı	ADS/I2C/GPIO durumu	sensör bağlantı sürekliliği
Ölçüm aktarımı	sensor_readings satırları	zaman damgalı değerler
Cihaz sağlığı	pool_system_status	RSSI, gerilim, son senkron
Alarm davranışı	sensor_event_logs	event_type, severity, alarm_key
Panel kullanımı	grafik, kart, dışa aktarım	okunabilir kullanıcı çıktısı

Şekil 11. Doğrulama matrisi: doğrulama alanları, kanıt kaynakları ve beklenen çıktılar.

13. Sonuç ve Değerlendirme

Bu çalışmada, alg üretim havuzları için çok parametreli su kalitesi izleme ve uzaktan yönetim işlevlerini bütünleşik biçimde sağlayan bir prototip tasarlanmış ve gerçeklenmiştir. Geliştirilen sistem; ölçüm toplama, güvenli veri saklama, kullanıcıya bağlı çok havuzlu cihaz yönetimi, uzaktan eşik/alarm yapılandırması ve olay tabanlı uyarı üretimi yeteneklerini düşük maliyetli bileşenlerle bir araya getirmektedir. Katmanlı mimari, satır düzeyi güvenlik ve kalıcı yapılandırma önbelleği sayesinde sistem; çoklu kullanıcı, bağlantı kesintisi ve uzaktan yapılandırma senaryolarına karşı dayanıklı bir temel sunmaktadır.

İleriki çalışmalarda; analog probların saha kalibrasyonunun standart referans çözeltileriyle yapılması, uzun süreli kararlılık ve ölçüm belirsizliği analizlerinin yürütülmesi, enerji tüketiminin azaltılması için uyku kipi stratejilerinin değerlendirilmesi ve toplanan zaman serisi veriler üzerinde eğilim/anomali analizlerinin geliştirilmesi hedeflenmektedir.

14. Kaynakça

1. Espressif Systems, "ESP8266 Technical Reference," Teknik Doküman.
2. Texas Instruments, "ADS1115 Ultra-Small, Low-Power, 16-Bit Analog-to-Digital Converter," Veri Sayfası.
3. Maxim Integrated, "DS18B20 Programmable Resolution 1-Wire Digital Thermometer," Veri Sayfası.
4. Aosong, "DHT22 (AM2302) Digital Temperature and Humidity Sensor," Veri Sayfası.
5. Supabase, "PostgREST ve Row Level Security Dokümantasyonu."
6. PostgreSQL Global Development Group, "PostgreSQL Documentation."

Ek A — Gömülü Yazılım Kaynak Kodu

ESP8266 kenar düğümü üzerinde çalışan firmware kaynak kodu aşağıda verilmiştir. Wi-Fi parolası ve servis rolü anahtarı gibi gizli bilgiler güvenlik gerekçesiyle yer tutucu ile değiştirilmiştir.

ziraat123.ino

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClientSecure.h>
#include <ArduinoJson.h>
#include <Wire.h>
#include <Adafruit_ADS1X15.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <DHT.h>
#include <EEPROM.h>
#include <string.h>
#include <stddef.h>

// Dahili besleme voltajini olcmek icin (analog A0 ADS1115'e tasindigi icin bos).
ADC_MODE(ADC_VCC);

// --- WiFi Bilgileri ---
const char* ssid = "Multisensor";
const char* password = "********";

// --- Supabase Proje Bilgileri ---
const char* SUPABASE_URL = "https://qmbsptypsnaxojzjrzeu.supabase.co";
const char* SERVICE_ROLE_KEY = "<SUPABASE_SERVICE_ROLE_KEY>";

// --- Havuz Kimliği ---
const char* POOL_ID = "8668a7bf-9c21-4e46-940b-4cd81f3a7f6a";

// --- Sensör Pinleri ve Ayarları ---
#define DHTPIN   D6
#define DHTTYPE  DHT22
#define ONE_WIRE_BUS D5

Adafruit_ADS1115 ads;                  // I2C analog cevirici (pH, TDS, DO, LDR)
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature waterTempSens(&oneWire);
DHT dht(DHTPIN, DHTTYPE);

bool adsOk = false;                    // ADS1115 baslatildi mi

// --- Uyari / durum pinleri ---
#define RED_LED_PIN    D8              // WiFi baglanma / baglanti yok
#define GREEN_LED_PIN  D7              // WiFi baglandi
#define BUZZER_PIN     D0              // Esik disi sensor uyarisi

// --- WiFi durum zamanlamasi ---
const unsigned long WIFI_START_DELAY_MS = 5000;     // Acilistan 5 sn sonra
baglanmayi dene
```

```
const unsigned long WIFI_RETRY_INTERVAL_MS = 15000; // Baglanamazsa tekrar dene
const unsigned long WIFI_BLINK_INTERVAL_MS = 500;
bool wifiAttemptStarted = false;
unsigned long bootMillis = 0;
unsigned long lastWiFiAttempt = 0;
unsigned long lastWiFiBlink = 0;
bool redLedState = true;

// --- Gonderim Zamanlamasi ---
unsigned long previousMillis = 0;
const unsigned long DEFAULT_READING_INTERVAL_MS = 10000;
const unsigned long MIN_READING_INTERVAL_MS = 1000;
const unsigned long MAX_READING_INTERVAL_MS = 60000;
unsigned long readingIntervalMs = DEFAULT_READING_INTERVAL_MS;

// --- Site kontrollu alarm ayarlari ---
enum SensorSlot {
  SENSOR_PH,
  SENSOR_EC,
  SENSOR_TDS,
  SENSOR_DO,
  SENSOR_LDR,
  SENSOR_WATER_TEMP,
  SENSOR_AIR_TEMP,
  SENSOR_HUMIDITY,
  SENSOR_COUNT
};

const char* SENSOR_KEYS[SENSOR_COUNT] = {
  "ph", "ec", "tds", "do", "ldr", "waterTemp", "airTemp", "humidity"
};

struct SensorThreshold {
  float warningMin;
  float warningMax;
};

SensorThreshold alarmThresholds[SENSOR_COUNT] = {
  {6.5, 8.5},    // pH
  {1.0, 3.0},    // EC
  {150, 1200},   // TDS
  {5.0, 12.0},   // DO
  {20, 90},      // LDR
  {20, 30},      // Su sicakligi
  {18, 35},      // Hava sicakligi
  {40, 80}       // Nem
};

bool alarmSensorEnabled[SENSOR_COUNT] = {
  true, true, true, true, true, true, true, true
};
bool buzzerEnabled = true;
bool alarmActive = false;
String currentAlarmKey = "";
String mutedAlarmKey = "";
String lastReportedAlarmKey = "";
unsigned long lastSettingsFetch = 0;
const unsigned long SETTINGS_FETCH_INTERVAL_MS = 15000;

// --- Kalici ayar onbellegi ---
const size_t EEPROM_SIZE = 512;
```

```
const uint32_t SETTINGS_MAGIC = 0x5A495241; // ZIRA
const uint16_t SETTINGS_VERSION = 1;

struct PersistedSettings {
  uint32_t magic;
  uint16_t version;
  uint16_t size;
  uint32_t readingIntervalMs;
  uint8_t buzzerEnabled;
  uint8_t alarmSensorEnabled[SENSOR_COUNT];
  float warningMin[SENSOR_COUNT];
  float warningMax[SENSOR_COUNT];
  uint32_t checksum;
};

bool cachedSettingsLoaded = false;
unsigned long settingsSaveCount = 0;

// --- Buzzer zamanlamasi: 2 sn'de bir 0.5 sn ot ---
const unsigned long BUZZER_PERIOD_MS = 2000;
const unsigned long BUZZER_ON_MS = 500;
unsigned long lastBuzzerCycle = 0;
unsigned long buzzerStartedAt = 0;
bool buzzerOn = false;

// ========================================================================
void setup() {
  Serial.begin(115200);
  bootMillis = millis();

  pinMode(RED_LED_PIN, OUTPUT);
  pinMode(GREEN_LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(RED_LED_PIN, HIGH);    // Acilista kirmizi yanar
  digitalWrite(GREEN_LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  EEPROM.begin(EEPROM_SIZE);
  cachedSettingsLoaded = loadPersistedSettings();
  Serial.printf("Kalici ayar: %s\n", cachedSettingsLoaded ? "yuklendi" :
"varsayilan");

  // I2C (ESP8266): D2 = SDA, D1 = SCL
  Wire.begin(D2, D1);

  // ADS1115 baslat
  adsOk = ads.begin();
  if (adsOk) {
    ads.setGain(GAIN_TWOTHIRDS);       // +/-6.144V, 1 bit = 0.1875mV
    Serial.println(F("ADS1115 hazir."));
  } else {
    Serial.println(F("ADS1115 BULUNAMADI! Analog sensorler okunamayacak."));
  }

  waterTempSens.begin();
  dht.begin();

  WiFi.mode(WIFI_STA);
  Serial.println(F("Acilis beklemesi: WiFi 5 sn sonra denenecek."));
}
```

```
// ========================================================================
void loop() {
  unsigned long currentMillis = millis();

  handleWiFiStatus(currentMillis);

  if (currentMillis - previousMillis < readingIntervalMs) {
    handleBuzzer(currentMillis);
    return;
  }
  stopBuzzer();
  previousMillis = currentMillis;

  fetchPoolSettings(currentMillis);

  Serial.printf("Heap: %u, MaxBlock: %u\n", ESP.getFreeHeap(),
ESP.getMaxFreeBlockSize());

  // ------------------------------------------------------------------
  // 1. ANALOG SENSORLER (ADS1115)
  // ------------------------------------------------------------------
  float phVoltage = 0, tdsVoltage = 0, doVoltage = 0, ldrVoltage = 0;
  if (adsOk) {
    phVoltage  = ads.readADC_SingleEnded(0) * 0.1875 / 1000.0;
    tdsVoltage = ads.readADC_SingleEnded(1) * 0.1875 / 1000.0;
    doVoltage  = ads.readADC_SingleEnded(2) * 0.1875 / 1000.0;
    ldrVoltage = ads.readADC_SingleEnded(3) * 0.1875 / 1000.0;
  }

  // ------------------------------------------------------------------
  // 2. DIGER SENSORLER
  // ------------------------------------------------------------------
  waterTempSens.requestTemperatures();
  float wTemp = waterTempSens.getTempCByIndex(0);
  float aTemp = dht.readTemperature();
  float hum   = dht.readHumidity();

  // ------------------------------------------------------------------
  // 3. FIZIKSEL DEGER HESAPLARI
  // ------------------------------------------------------------------
  float phValue = 7.7 + (phVoltage - 1.30) * 4.0;
  phValue = constrain(phValue, 0, 14);

  float compTemp  = (wTemp == -127.00) ? 25.0 : wTemp;
  float compCoef  = 1.0 + 0.02 * (compTemp - 25.0);
  float compVolt  = tdsVoltage / compCoef;
  float tdsValue  = (133.42 * pow(compVolt, 3) - 255.86 * pow(compVolt, 2) + 857.39 *
compVolt) * 0.5;
  if (tdsValue < 0) tdsValue = 0;

  float ecValue   = tdsValue / 500.0;          // ppm -> mS/cm (yaklasik)
  float doValue   = doVoltage * 4.5;           // mg/L
  float ldrPercent = (ldrVoltage / 5.0) * 100.0;
  ldrPercent = constrain(ldrPercent, 0, 100);

  // ------------------------------------------------------------------
  // 4. SENSOR BAGLANTI (AKTIFLIK) DURUMLARI
  // ------------------------------------------------------------------
  bool waterTempOk = (wTemp != -127.00 && wTemp != 85.00);
  bool airTempOk   = !isnan(aTemp);
```

```
  bool humOk       = !isnan(hum);
  bool phOk  = adsOk;
  bool tdsOk = adsOk;
  bool ecOk  = adsOk;
  bool doOk  = adsOk;
  bool ldrOk = adsOk;

  updateAlarmState(phValue, ecValue, tdsValue, doValue, ldrPercent, wTemp, aTemp,
hum,
                   phOk, ecOk, tdsOk, doOk, ldrOk, waterTempOk, airTempOk, humOk);

  // ---------------------------------------------------------------
  // 5. SERIAL RAPORU
  // ---------------------------------------------------------------
  Serial.println(F("\n================================================"));
  Serial.printf ("  Uptime : %lu sn   WiFi: %s (RSSI %d dBm)   Vcc: %.2f V\n",
                 millis() / 1000,
                 WiFi.status() == WL_CONNECTED ? "BAGLI" : "YOK",
                 WiFi.RSSI(),
                 ESP.getVcc() / 1000.0);
  Serial.println(F("------------------------------------------------"));
  Serial.printf ("  pH    : %.2f  (%.3f V) [%s]\n", phValue, phVoltage, phOk ? "OK" :
"YOK");
  Serial.printf ("  TDS   : %.1f ppm  [%s]\n", tdsValue, tdsOk ? "OK" : "YOK");
  Serial.printf ("  EC    : %.2f mS/cm\n", ecValue);
  Serial.printf ("  DO    : %.2f mg/L  [%s]\n", doValue, doOk ? "OK" : "YOK");
  Serial.printf ("  LDR   : %% %.1f  [%s]\n", ldrPercent, ldrOk ? "OK" : "YOK");
  Serial.printf ("  Su Sic: %s\n", waterTempOk ? String(wTemp, 2).c_str() : "BAGLANTI
HATASI");
  Serial.printf ("  Hava  : %s\n", airTempOk ? String(aTemp, 2).c_str() : "BAGLANTI
HATASI");
  Serial.printf ("  Nem   : %s\n", humOk ? String(hum, 2).c_str() : "BAGLANTI
HATASI");
  Serial.println(F("================================================"));

  // ---------------------------------------------------------------
  // 6. SUPABASE'E GONDER
  // ---------------------------------------------------------------
  if (WiFi.status() == WL_CONNECTED) {
    sendSensorReadings(phValue, ecValue, tdsValue, doValue, ldrPercent, wTemp, aTemp,
hum,
                       phOk, ecOk, tdsOk, doOk, ldrOk, waterTempOk, airTempOk,
humOk);
    updateSystemStatus(phOk, ecOk, tdsOk, doOk, ldrOk, waterTempOk, airTempOk, humOk,
                       phVoltage, tdsVoltage, doVoltage, ldrVoltage);
  } else {
    Serial.println(F("  >> WiFi yok, gonderim atlandi."));
  }
}

// ===========================================================================
void startWiFiAttempt(unsigned long now) {
  WiFi.disconnect();
  WiFi.begin(ssid, password);
  wifiAttemptStarted = true;
  lastWiFiAttempt = now;
  Serial.print(F("WiFi'ye baglaniyor"));
}

void handleWiFiStatus(unsigned long now) {
  if (now - bootMillis < WIFI_START_DELAY_MS) {
```

```
    digitalWrite(RED_LED_PIN, HIGH);
    digitalWrite(GREEN_LED_PIN, LOW);
    return;
  }

  if (WiFi.status() == WL_CONNECTED) {
    digitalWrite(RED_LED_PIN, LOW);
    digitalWrite(GREEN_LED_PIN, HIGH);
    return;
  }

  digitalWrite(GREEN_LED_PIN, LOW);
  if (now - lastWiFiBlink >= WIFI_BLINK_INTERVAL_MS) {
    lastWiFiBlink = now;
    redLedState = !redLedState;
    digitalWrite(RED_LED_PIN, redLedState ? HIGH : LOW);
    Serial.print(F("."));
  }

  if (!wifiAttemptStarted || now - lastWiFiAttempt >= WIFI_RETRY_INTERVAL_MS) {
    startWiFiAttempt(now);
  }
}

bool isOutsideWarning(int index, float value) {
  return value < alarmThresholds[index].warningMin || value >
alarmThresholds[index].warningMax;
}

String buildAlarmPart(int index, float value) {
  String part = SENSOR_KEYS[index];
  part += ":";
  part += value < alarmThresholds[index].warningMin ? "low" : "high";
  return part;
}

void updateAlarmState(float ph, float ec, float tds, float dox, float ldr,
                      float wTemp, float aTemp, float hum,
                      bool phOk, bool ecOk, bool tdsOk, bool doOk, bool ldrOk,
                      bool wTempOk, bool aTempOk, bool humOk) {
  float values[SENSOR_COUNT] = { ph, ec, tds, dox, ldr, wTemp, aTemp, hum };
  bool ok[SENSOR_COUNT] = { phOk, ecOk, tdsOk, doOk, ldrOk, wTempOk, aTempOk, humOk
};

  String nextAlarmKey = "";
  alarmActive = false;
  for (int i = 0; i < SENSOR_COUNT; i++) {
    if (!alarmSensorEnabled[i] || !ok[i]) continue;
    if (isOutsideWarning(i, values[i])) {
      alarmActive = true;
      if (nextAlarmKey.length() > 0) nextAlarmKey += "|";
      nextAlarmKey += buildAlarmPart(i, values[i]);
      Serial.printf("  [UYARI] %s esik disi: %.2f (%.2f - %.2f)\n",
                    SENSOR_KEYS[i], values[i], alarmThresholds[i].warningMin,
alarmThresholds[i].warningMax);
    }
  }

  currentAlarmKey = nextAlarmKey;
  syncAlarmTransition();
}
```

```
void stopBuzzer() {
  digitalWrite(BUZZER_PIN, LOW);
  buzzerOn = false;
}

void handleBuzzer(unsigned long now) {
  bool mutedForCurrentAlarm = currentAlarmKey.length() > 0 && mutedAlarmKey ==
currentAlarmKey;
  bool shouldBuzz = alarmActive && buzzerEnabled && !mutedForCurrentAlarm;
  if (!shouldBuzz) {
    stopBuzzer();
    return;
  }

  if (!buzzerOn && now - lastBuzzerCycle >= BUZZER_PERIOD_MS) {
    digitalWrite(BUZZER_PIN, HIGH);
    buzzerOn = true;
    buzzerStartedAt = now;
  }

  if (buzzerOn && now - buzzerStartedAt >= BUZZER_ON_MS) {
    digitalWrite(BUZZER_PIN, LOW);
    buzzerOn = false;
    lastBuzzerCycle = now;
  }
}

void fetchPoolSettings(unsigned long now) {
  if (WiFi.status() != WL_CONNECTED) return;
  if (lastSettingsFetch != 0 && now - lastSettingsFetch < SETTINGS_FETCH_INTERVAL_MS)
return;
  lastSettingsFetch = now;

  WiFiClientSecure client;
  client.setInsecure();
  client.setBufferSizes(4096, 512);
  HTTPClient http;
  http.setReuse(false);
  http.setTimeout(10000);

  String url = String(SUPABASE_URL) + "/rest/v1/pools?id=eq." + POOL_ID +
"&select=alarm_settings,sensor_thresholds";
  http.begin(client, url);
  addCommonHeaders(http);

  int code = http.GET();
  if (code < 200 || code >= 300) {
    Serial.printf("  [HATA] Alarm Ayarlari -> HTTP %d | %s\n", code,
http.errorToString(code).c_str());
    Serial.println(http.getString());
    http.end();
    return;
  }

  DynamicJsonDocument doc(3072);
  DeserializationError error = deserializeJson(doc, http.getString());
  http.end();
  if (error) {
    Serial.printf("  [HATA] Alarm Ayarlari JSON -> %s\n", error.c_str());
    return;
```

```
  }

  JsonArray rows = doc.as<JsonArray>();
  if (rows.isNull() || rows.size() == 0) return;

  JsonObject row = rows[0].as<JsonObject>();
  bool settingsChanged = false;
  JsonObject alarm = row["alarm_settings"].as<JsonObject>();
  if (!alarm.isNull()) {
    if (alarm.containsKey("buzzerEnabled")) {
      bool nextBuzzerEnabled = alarm["buzzerEnabled"].as<bool>();
      if (buzzerEnabled != nextBuzzerEnabled) {
        buzzerEnabled = nextBuzzerEnabled;
        settingsChanged = true;
      }
    }
    if (alarm.containsKey("readingIntervalMs")) {
      unsigned long nextInterval =
clampReadingInterval(alarm["readingIntervalMs"].as<unsigned long>());
      if (readingIntervalMs != nextInterval) {
        readingIntervalMs = nextInterval;
        settingsChanged = true;
      }
    }
    if (alarm.containsKey("mutedAlarmKey") && !alarm["mutedAlarmKey"].isNull()) {
      mutedAlarmKey = alarm["mutedAlarmKey"].as<String>();
    } else {
      mutedAlarmKey = "";
    }

    JsonObject sensors = alarm["alarmSensors"].as<JsonObject>();
    if (!sensors.isNull()) {
      for (int i = 0; i < SENSOR_COUNT; i++) {
        if (sensors.containsKey(SENSOR_KEYS[i])) {
          bool nextEnabled = sensors[SENSOR_KEYS[i]].as<bool>();
          if (alarmSensorEnabled[i] != nextEnabled) {
            alarmSensorEnabled[i] = nextEnabled;
            settingsChanged = true;
          }
        }
      }
    }
  }

  JsonObject thresholds = row["sensor_thresholds"].as<JsonObject>();
  if (!thresholds.isNull()) {
    for (int i = 0; i < SENSOR_COUNT; i++) {
      JsonObject t = thresholds[SENSOR_KEYS[i]].as<JsonObject>();
      if (t.isNull()) continue;
      if (t.containsKey("warningMin")) {
        float nextMin = t["warningMin"].as<float>();
        if (fabs(alarmThresholds[i].warningMin - nextMin) > 0.001) {
          alarmThresholds[i].warningMin = nextMin;
          settingsChanged = true;
        }
      }
      if (t.containsKey("warningMax")) {
        float nextMax = t["warningMax"].as<float>();
        if (fabs(alarmThresholds[i].warningMax - nextMax) > 0.001) {
          alarmThresholds[i].warningMax = nextMax;
          settingsChanged = true;
```

```
        }
      }
    }
  }

  if (settingsChanged || !cachedSettingsLoaded) {
    if (savePersistedSettings()) {
      cachedSettingsLoaded = true;
    }
  }

  Serial.printf("  [OK]   Alarm Ayarlari -> buzzer:%s mute:%s alarm:%s interval:%lu
ms\n",
                buzzerEnabled ? "acik" : "kapali",
                mutedAlarmKey.length() > 0 ? mutedAlarmKey.c_str() : "yok",
                alarmActive ? "aktif" : "normal",
                readingIntervalMs);
}

unsigned long clampReadingInterval(unsigned long value) {
  if (value < MIN_READING_INTERVAL_MS) return MIN_READING_INTERVAL_MS;
  if (value > MAX_READING_INTERVAL_MS) return MAX_READING_INTERVAL_MS;
  return value;
}

uint32_t computeSettingsChecksumBytes(const uint8_t* bytes, size_t length) {
  uint32_t hash = 2166136261UL;
  for (size_t i = 0; i < length; i++) {
    hash ^= bytes[i];
    hash *= 16777619UL;
  }
  return hash;
}

bool loadPersistedSettings() {
  PersistedSettings settings;
  EEPROM.get(0, settings);

  if (settings.magic != SETTINGS_MAGIC ||
      settings.version != SETTINGS_VERSION ||
      settings.size != sizeof(PersistedSettings) ||
      settings.checksum != computeSettingsChecksumBytes(
        reinterpret_cast<const uint8_t*>(&settings),
        offsetof(PersistedSettings, checksum)
      )) {
    return false;
  }

  readingIntervalMs = clampReadingInterval(settings.readingIntervalMs);
  buzzerEnabled = settings.buzzerEnabled == 1;
  for (int i = 0; i < SENSOR_COUNT; i++) {
    alarmSensorEnabled[i] = settings.alarmSensorEnabled[i] == 1;
    if (isfinite(settings.warningMin[i]) && isfinite(settings.warningMax[i])) {
      alarmThresholds[i].warningMin = settings.warningMin[i];
      alarmThresholds[i].warningMax = settings.warningMax[i];
    }
  }
  return true;
}

bool savePersistedSettings() {
```

```
  PersistedSettings settings;
  memset(&settings, 0, sizeof(settings));
  settings.magic = SETTINGS_MAGIC;
  settings.version = SETTINGS_VERSION;
  settings.size = sizeof(PersistedSettings);
  settings.readingIntervalMs = readingIntervalMs;
  settings.buzzerEnabled = buzzerEnabled ? 1 : 0;
  for (int i = 0; i < SENSOR_COUNT; i++) {
    settings.alarmSensorEnabled[i] = alarmSensorEnabled[i] ? 1 : 0;
    settings.warningMin[i] = alarmThresholds[i].warningMin;
    settings.warningMax[i] = alarmThresholds[i].warningMax;
  }
  settings.checksum = computeSettingsChecksumBytes(
    reinterpret_cast<const uint8_t*>(&settings),
    offsetof(PersistedSettings, checksum)
  );

  EEPROM.put(0, settings);
  bool ok = EEPROM.commit();
  if (ok) {
    settingsSaveCount++;
    Serial.println(F("  [OK]   Kalici ayar onbellegi guncellendi."));
  } else {
    Serial.println(F("  [HATA] Kalici ayar onbellegi yazilamadi."));
  }
  return ok;
}

void syncAlarmTransition() {
  if (WiFi.status() != WL_CONNECTED) return;

  if (currentAlarmKey != lastReportedAlarmKey) {
    if (currentAlarmKey.length() > 0) {
      postEventLog("alarm_active", currentAlarmKey, "Sensor esik disi alarmi
basladi.");
    } else if (lastReportedAlarmKey.length() > 0) {
      postEventLog("alarm_resolved", lastReportedAlarmKey, "Sensor esik disi alarmi
normale dondu.");
    }
    lastReportedAlarmKey = currentAlarmKey;
  }

  if (currentAlarmKey.length() > 0 && mutedAlarmKey.length() > 0 && mutedAlarmKey !=
currentAlarmKey) {
    mutedAlarmKey = "";
    patchAlarmMute("");
  }

  if (currentAlarmKey.length() == 0 && mutedAlarmKey.length() > 0) {
    mutedAlarmKey = "";
    patchAlarmMute("");
  }
}

void patchAlarmMute(const String &muteKey) {
  WiFiClientSecure client;
  client.setInsecure();
  client.setBufferSizes(4096, 512);
  HTTPClient http;
  http.setReuse(false);
  http.setTimeout(10000);
```

```
  String url = String(SUPABASE_URL) + "/rest/v1/pools?id=eq." + POOL_ID;
  http.begin(client, url);
  addCommonHeaders(http);

  StaticJsonDocument<768> doc;
  JsonObject alarm = doc.createNestedObject("alarm_settings");
  alarm["buzzerEnabled"] = buzzerEnabled;
  alarm["buzzerMuted"] = false;
  alarm["readingIntervalMs"] = (int)readingIntervalMs;
  if (muteKey.length() > 0) {
    alarm["mutedAlarmKey"] = muteKey;
    alarm["mutedAt"] = String(millis() / 1000);
  } else {
    alarm["mutedAlarmKey"] = nullptr;
    alarm["mutedAt"] = nullptr;
  }

  JsonObject sensors = alarm.createNestedObject("alarmSensors");
  for (int i = 0; i < SENSOR_COUNT; i++) {
    sensors[SENSOR_KEYS[i]] = alarmSensorEnabled[i];
  }

  String body;
  serializeJson(doc, body);
  int code = http.PATCH(body);
  if (code < 200 || code >= 300) {
    Serial.printf("  [HATA] Alarm mute temizleme -> HTTP %d | %s\n", code,
http.errorToString(code).c_str());
    Serial.println(http.getString());
  }
  http.end();
}

void postEventLog(const char* eventType, const String &alarmKey, const char* message)
{
  WiFiClientSecure client;
  client.setInsecure();
  client.setBufferSizes(4096, 512);
  HTTPClient http;
  http.setReuse(false);
  http.setTimeout(10000);

  String url = String(SUPABASE_URL) + "/rest/v1/sensor_event_logs";
  http.begin(client, url);
  addCommonHeaders(http);

  StaticJsonDocument<768> doc;
  doc["pool_id"] = POOL_ID;
  doc["event_type"] = eventType;
  doc["alarm_key"] = alarmKey;
  doc["severity"] = strcmp(eventType, "alarm_active") == 0 ? "warning" : "info";
  doc["message"] = message;

  JsonObject meta = doc.createNestedObject("metadata");
  meta["uptimeSeconds"] = (int)(millis() / 1000);
  meta["rssi"] = (int)WiFi.RSSI();
  meta["activeAlarmKey"] = currentAlarmKey;
  meta["mutedAlarmKey"] = mutedAlarmKey;

  String body;
```

```
  serializeJson(doc, body);
  int code = http.POST(body);
  if (code < 200 || code >= 300) {
    Serial.printf("  [HATA] Olay Logu -> HTTP %d | %s\n", code,
http.errorToString(code).c_str());
    Serial.println(http.getString());
  }
  http.end();
}

void addCommonHeaders(HTTPClient &http) {
  http.addHeader("Content-Type", "application/json");
  http.addHeader("apikey", SERVICE_ROLE_KEY);
  http.addHeader("Authorization", String("Bearer ") + SERVICE_ROLE_KEY);
  http.addHeader("Prefer", "return=minimal");
}

// Sistem durumu + sensor aktiflik bayraklari (PATCH)
void updateSystemStatus(bool ph, bool ec, bool tds, bool dox, bool ldr,
                        bool wTemp, bool aTemp, bool hum,
                        float phV, float tdsV, float doV, float ldrV) {
  WiFiClientSecure client;
  client.setInsecure();
  client.setBufferSizes(4096, 512);
  HTTPClient http;
  http.setReuse(false);
  http.setTimeout(15000);

  String url = String(SUPABASE_URL) + "/rest/v1/pool_system_status?pool_id=eq." +
POOL_ID;
  http.begin(client, url);
  addCommonHeaders(http);

  StaticJsonDocument<768> doc;
  doc["uptime_seconds"] = (int)(millis() / 1000);
  doc["rssi"] = (int)WiFi.RSSI();
  doc["voltage"] = serialized(String(ESP.getVcc() / 1000.0, 2));

  JsonObject conn = doc.createNestedObject("sensor_connections");
  conn["ph"]        = ph;
  conn["ec"]        = ec;
  conn["tds"]       = tds;
  conn["do"]        = dox;
  conn["ldr"]       = ldr;
  conn["waterTemp"] = wTemp;
  conn["airTemp"]   = aTemp;
  conn["humidity"]  = hum;

  // Gelistirici tani verisi (sitede dev modu acikken gorunur)
  JsonObject diag = doc.createNestedObject("diagnostics");
  diag["freeHeap"] = ESP.getFreeHeap();
  diag["maxBlock"] = ESP.getMaxFreeBlockSize();
  diag["adsOk"]    = adsOk;
  diag["activeAlarmKey"] = currentAlarmKey;
  diag["readingIntervalMs"] = (int)readingIntervalMs;
  diag["cachedSettingsLoaded"] = cachedSettingsLoaded;
  diag["settingsSaveCount"] = (int)settingsSaveCount;
  JsonObject rawV  = diag.createNestedObject("rawVoltage");
  rawV["ph"]  = serialized(String(phV, 3));
  rawV["tds"] = serialized(String(tdsV, 3));
  rawV["do"]  = serialized(String(doV, 3));
```

```
  rawV["ldr"] = serialized(String(ldrV, 3));

  String body;
  serializeJson(doc, body);
  int code = http.PATCH(body);

  if (code >= 200 && code < 300) {
    Serial.printf("  [OK]   Sistem Durumu  -> HTTP %d\n", code);
  } else {
    Serial.printf("  [HATA] Sistem Durumu  -> HTTP %d | %s\n", code,
http.errorToString(code).c_str());
    Serial.println(http.getString());
  }
  http.end();
}

// Sensor olcumlerini toplu gonder (POST). Sadece bagli sensorler gonderilir.
void sendSensorReadings(float ph, float ec, float tds, float dox, float ldr,
                        float wTemp, float aTemp, float hum,
                        bool phOk, bool ecOk, bool tdsOk, bool doOk, bool ldrOk,
                        bool wTempOk, bool aTempOk, bool humOk) {
  WiFiClientSecure client;
  client.setInsecure();
  client.setBufferSizes(4096, 512);
  HTTPClient http;
  http.setReuse(false);
  http.setTimeout(15000);

  String url = String(SUPABASE_URL) + "/rest/v1/sensor_readings";
  http.begin(client, url);
  addCommonHeaders(http);

  DynamicJsonDocument doc(1024);
  addReading(doc, "ph",        ph,  phOk);
  addReading(doc, "ec",        ec,  ecOk);
  addReading(doc, "tds",       tds, tdsOk);
  addReading(doc, "do",        dox, doOk);
  addReading(doc, "ldr",       ldr, ldrOk);
  addReading(doc, "waterTemp", wTemp, wTempOk);
  addReading(doc, "airTemp",   aTemp, aTempOk);
  addReading(doc, "humidity",  hum, humOk);

  String body;
  serializeJson(doc, body);
  int code = http.POST(body);

  if (code >= 200 && code < 300) {
    Serial.printf("  [OK]   Sensor Verisi  -> HTTP %d\n", code);
  } else {
    Serial.printf("  [HATA] Sensor Verisi  -> HTTP %d | %s\n", code,
http.errorToString(code).c_str());
    Serial.println(http.getString());
  }
  http.end();
}

// Bagli olmayan sensoru atla; degeri numeric(10,2) hassasiyetle ekle
void addReading(JsonDocument &doc, const char* id, float value, bool ok) {
  if (!ok) return;
  JsonObject obj = doc.createNestedObject();
  obj["pool_id"]   = POOL_ID;
```

```
  obj["sensor_id"] = id;
  obj["value"]     = serialized(String(value, 2));
}
```

Ek B — Veritabanı Şeması (SQL)

Veri katmanının tablo tanımları, kısıtlar, satır düzeyi güvenlik politikaları ve yardımcı fonksiyonları içeren göç (migration) betikleri sırasıyla aşağıda verilmiştir.

20260314235500_init_aico_workspace.sql

```
create extension if not exists pgcrypto;

create or replace function public.touch_updated_at()
returns trigger
language plpgsql
as $$
begin
  new.updated_at = timezone('utc', now());
  return new;
end;
$$;

create table if not exists public.user_preferences (
  user_id uuid primary key references auth.users(id) on delete cascade,
  use_mock_data boolean not null default true,
  refresh_interval_ms integer not null default 3000 check (refresh_interval_ms
between 1000 and 60000),
  created_at timestamptz not null default timezone('utc', now()),
  updated_at timestamptz not null default timezone('utc', now())
);

create table if not exists public.pools (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  name text not null,
  location text not null default '',
  sort_order integer not null default 0,
  sensor_offsets jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default timezone('utc', now()),
  unique (user_id, name)
);

create table if not exists public.pool_system_status (
  pool_id uuid primary key references public.pools(id) on delete cascade,
  uptime_seconds integer not null default 0,
  rssi integer not null default -60,
  voltage numeric(4, 2) not null default 3.70,
  last_sync timestamptz not null default timezone('utc', now()),
  sensor_connections jsonb not null default jsonb_build_object(
    'ph', true,
    'ec', true,
    'do', true,
    'waterTemp', true,
    'airTemp', true,
    'humidity', true
  ),
  created_at timestamptz not null default timezone('utc', now()),
  updated_at timestamptz not null default timezone('utc', now())
);
```

```
create table if not exists public.sensor_readings (
  id bigint generated always as identity primary key,
  pool_id uuid not null references public.pools(id) on delete cascade,
  sensor_id text not null check (sensor_id in ('ph', 'ec', 'do', 'waterTemp',
'airTemp', 'humidity')),
  value numeric(10, 2) not null,
  recorded_at timestamptz not null default timezone('utc', now())
);

create index if not exists sensor_readings_pool_sensor_time_idx
  on public.sensor_readings (pool_id, sensor_id, recorded_at desc);

drop trigger if exists set_user_preferences_updated_at on public.user_preferences;
create trigger set_user_preferences_updated_at
before update on public.user_preferences
for each row
execute function public.touch_updated_at();

drop trigger if exists set_pool_system_status_updated_at on
public.pool_system_status;
create trigger set_pool_system_status_updated_at
before update on public.pool_system_status
for each row
execute function public.touch_updated_at();

alter table public.user_preferences enable row level security;
alter table public.pools enable row level security;
alter table public.pool_system_status enable row level security;
alter table public.sensor_readings enable row level security;

create policy "user_preferences_select_own"
on public.user_preferences
for select
to authenticated
using (auth.uid() = user_id);

create policy "user_preferences_insert_own"
on public.user_preferences
for insert
to authenticated
with check (auth.uid() = user_id);

create policy "user_preferences_update_own"
on public.user_preferences
for update
to authenticated
using (auth.uid() = user_id)
with check (auth.uid() = user_id);

create policy "pools_select_own"
on public.pools
for select
to authenticated
using (auth.uid() = user_id);

create policy "pools_insert_own"
on public.pools
for insert
to authenticated
with check (auth.uid() = user_id);
```

```
create policy "pools_update_own"
on public.pools
for update
to authenticated
using (auth.uid() = user_id)
with check (auth.uid() = user_id);

create policy "pool_system_status_select_own"
on public.pool_system_status
for select
to authenticated
using (
  exists (
    select 1
    from public.pools
    where pools.id = pool_system_status.pool_id
      and pools.user_id = auth.uid()
  )
);

create policy "pool_system_status_insert_own"
on public.pool_system_status
for insert
to authenticated
with check (
  exists (
    select 1
    from public.pools
    where pools.id = pool_system_status.pool_id
      and pools.user_id = auth.uid()
  )
);

create policy "pool_system_status_update_own"
on public.pool_system_status
for update
to authenticated
using (
  exists (
    select 1
    from public.pools
    where pools.id = pool_system_status.pool_id
      and pools.user_id = auth.uid()
  )
)
with check (
  exists (
    select 1
    from public.pools
    where pools.id = pool_system_status.pool_id
      and pools.user_id = auth.uid()
  )
);

create policy "sensor_readings_select_own"
on public.sensor_readings
for select
to authenticated
using (
  exists (
```

```
    select 1
    from public.pools
    where pools.id = sensor_readings.pool_id
      and pools.user_id = auth.uid()
  )
);

create policy "sensor_readings_insert_own"
on public.sensor_readings
for insert
to authenticated
with check (
  exists (
    select 1
    from public.pools
    where pools.id = sensor_readings.pool_id
      and pools.user_id = auth.uid()
  )
);

create policy "sensor_readings_update_own"
on public.sensor_readings
for update
to authenticated
using (
  exists (
    select 1
    from public.pools
    where pools.id = sensor_readings.pool_id
      and pools.user_id = auth.uid()
  )
)
with check (
  exists (
    select 1
    from public.pools
    where pools.id = sensor_readings.pool_id
      and pools.user_id = auth.uid()
  )
);

create or replace function public.bootstrap_user_workspace()
returns void
language plpgsql
security definer
set search_path = public
as $$
begin
  if auth.uid() is null then
    raise exception 'Authentication required';
  end if;

  insert into public.user_preferences (user_id)
  values (auth.uid())
  on conflict (user_id) do nothing;

  if not exists (
    select 1
    from public.pools
    where user_id = auth.uid()
  ) then
```

```
    insert into public.pools (user_id, name, location, sort_order, sensor_offsets)
    values
      (
        auth.uid(),
        'Alpha-1',
        'Lab B, Rack 3',
        1,
        jsonb_build_object('ph', 0, 'ec', 0, 'do', 0, 'waterTemp', 0, 'airTemp', 0,
'humidity', 0)
      ),
      (
        auth.uid(),
        'Beta-2',
        'Lab A, Rack 1',
        2,
        jsonb_build_object('ph', 0.3, 'ec', -0.2, 'do', 1.5, 'waterTemp', 2,
'airTemp', -1, 'humidity', -5)
      ),
      (
        auth.uid(),
        'Gamma-3',
        'Greenhouse, Section C',
        3,
        jsonb_build_object('ph', -0.5, 'ec', 0.4, 'do', -0.8, 'waterTemp', 3.5,
'airTemp', 4, 'humidity', 10)
      );
  end if;

  insert into public.pool_system_status (pool_id)
  select pools.id
  from public.pools
  where pools.user_id = auth.uid()
  on conflict (pool_id) do nothing;
end;
$$;

grant execute on function public.bootstrap_user_workspace() to authenticated;
```

20260317004000_default_live_mode.sql

```
alter table public.user_preferences
alter column use_mock_data set default false;

update public.user_preferences
set
  use_mock_data = false,
  updated_at = timezone('utc', now())
where use_mock_data = true;
```

20260317023000_auth_bootstrap_and_pool_management.sql

```
create or replace function public.default_sensor_offsets()
returns jsonb
language sql
immutable
as $$
  select jsonb_build_object(
```

```
    'ph', 0,
    'ec', 0,
    'do', 0,
    'waterTemp', 0,
    'airTemp', 0,
    'humidity', 0
  );
$$;

create or replace function public.ensure_user_workspace(target_user_id uuid)
returns uuid
language plpgsql
security definer
set search_path = public
as $$
declare
  initial_pool_id uuid;
begin
  if target_user_id is null then
    raise exception 'target_user_id is required';
  end if;

  insert into public.user_preferences (user_id)
  values (target_user_id)
  on conflict (user_id) do nothing;

  select pools.id
  into initial_pool_id
  from public.pools
  where pools.user_id = target_user_id
  order by pools.sort_order asc, pools.created_at asc
  limit 1;

  if initial_pool_id is null then
    insert into public.pools (user_id, name, location, sort_order, sensor_offsets)
    values (
      target_user_id,
      'Pool-1',
      'Yeni kayit havuzu',
      1,
      public.default_sensor_offsets()
    )
    returning id into initial_pool_id;
  end if;

  insert into public.pool_system_status (pool_id)
  select pools.id
  from public.pools
  where pools.user_id = target_user_id
  on conflict (pool_id) do nothing;

  return initial_pool_id;
end;
$$;

create or replace function public.bootstrap_user_workspace()
returns void
language plpgsql
security definer
set search_path = public
as $$
```

```
begin
  if auth.uid() is null then
    raise exception 'Authentication required';
  end if;

  perform public.ensure_user_workspace(auth.uid());
end;
$$;

create or replace function public.handle_new_auth_user()
returns trigger
language plpgsql
security definer
set search_path = public
as $$
begin
  perform public.ensure_user_workspace(new.id);
  return new;
end;
$$;

drop trigger if exists on_auth_user_created on auth.users;
create trigger on_auth_user_created
after insert on auth.users
for each row
execute function public.handle_new_auth_user();

create or replace function public.create_pool(pool_name text, pool_location text
default '')
returns uuid
language plpgsql
security definer
set search_path = public
as $$
declare
  trimmed_name text;
  next_sort integer;
  new_pool_id uuid;
begin
  if auth.uid() is null then
    raise exception 'Authentication required';
  end if;

  trimmed_name := nullif(btrim(pool_name), '');
  if trimmed_name is null then
    raise exception 'Pool name is required';
  end if;

  select coalesce(max(pools.sort_order), 0) + 1
  into next_sort
  from public.pools
  where pools.user_id = auth.uid();

  insert into public.pools (user_id, name, location, sort_order, sensor_offsets)
  values (
    auth.uid(),
    trimmed_name,
    coalesce(nullif(btrim(pool_location), ''), ''),
    next_sort,
    public.default_sensor_offsets()
  )
```

```
  returning id into new_pool_id;

  insert into public.pool_system_status (pool_id)
  values (new_pool_id)
  on conflict (pool_id) do nothing;

  return new_pool_id;
end;
$$;

revoke all on function public.create_pool(text, text) from public;
grant execute on function public.create_pool(text, text) to authenticated;
```

20260319000000_clean_default_pool_location.sql

```
-- Clean placeholder "Yeni kayit havuzu" location from existing rows and from
bootstrap function.
update public.pools
   set location = ''
 where location = 'Yeni kayit havuzu';

create or replace function public.ensure_user_workspace(target_user_id uuid)
returns uuid
language plpgsql
security definer
set search_path = public
as $$
declare
  initial_pool_id uuid;
begin
  if target_user_id is null then
    raise exception 'target_user_id is required';
  end if;

  insert into public.user_preferences (user_id)
  values (target_user_id)
  on conflict (user_id) do nothing;

  select pools.id
  into initial_pool_id
  from public.pools
  where pools.user_id = target_user_id
  order by pools.sort_order asc, pools.created_at asc
  limit 1;

  if initial_pool_id is null then
    insert into public.pools (user_id, name, location, sort_order, sensor_offsets)
    values (
      target_user_id,
      'Pool-1',
      '',
      1,
      public.default_sensor_offsets()
    )
    returning id into initial_pool_id;
  end if;

  insert into public.pool_system_status (pool_id)
  select pools.id
```

```
  from public.pools
  where pools.user_id = target_user_id
  on conflict (pool_id) do nothing;

  return initial_pool_id;
end;
$$;
```

20260606000000_add_tds_ldr_sensors.sql

```
-- Yeni sensorler: TDS (ppm) ve LDR (isik %).
-- ESP firmware artik bu iki sensoru de gonderiyor.

-- 1) sensor_readings.sensor_id CHECK kisitini tds/ldr icerecek sekilde genislet
alter table public.sensor_readings
  drop constraint if exists sensor_readings_sensor_id_check;

alter table public.sensor_readings
  add constraint sensor_readings_sensor_id_check
  check (sensor_id in ('ph', 'ec', 'tds', 'do', 'ldr', 'waterTemp', 'airTemp',
'humidity'));

-- 2) pool_system_status.sensor_connections default'unu yeni sensorlerle guncelle
alter table public.pool_system_status
  alter column sensor_connections set default jsonb_build_object(
    'ph', true,
    'ec', true,
    'tds', true,
    'do', true,
    'ldr', true,
    'waterTemp', true,
    'airTemp', true,
    'humidity', true
  );

-- Mevcut satirlara eksik anahtarlari ekle (varsa degeri korunur)
update public.pool_system_status
set sensor_connections = jsonb_build_object('tds', true, 'ldr', true) ||
sensor_connections;

-- 3) default_sensor_offsets() fonksiyonuna tds/ldr ekle
create or replace function public.default_sensor_offsets()
returns jsonb
language sql
immutable
as $$
  select jsonb_build_object(
    'ph', 0,
    'ec', 0,
    'tds', 0,
    'do', 0,
    'ldr', 0,
    'waterTemp', 0,
    'airTemp', 0,
    'humidity', 0
  );
$$;

-- Mevcut havuzlarin offset'lerine eksik anahtarlari ekle
```

```
update public.pools
set sensor_offsets = jsonb_build_object('tds', 0, 'ldr', 0) || sensor_offsets;
```

20260606010000_pool_claim_model.sql

```
-- Pool sahiplenme (claim) modeli.
-- ESP cihazlari sabit bir pool_id ile gelir; havuz DB'de sahipsiz (user_id NULL)
-- durabilir. Kullanici site uzerinden pool_id girince havuz onun hesabina baglanir.

-- 1) pools.user_id artik NULL olabilir (sahipsiz cihaz havuzu)
alter table public.pools
  alter column user_id drop not null;

-- 2) Yeni kullaniciya otomatik havuz ACMA. Sadece tercih kaydi olusturulur;
--    kullanici kendi cihazini pool_id ile ekler.
create or replace function public.ensure_user_workspace(target_user_id uuid)
returns uuid
language plpgsql
security definer
set search_path = public
as $$
begin
  if target_user_id is null then
    raise exception 'target_user_id is required';
  end if;

  insert into public.user_preferences (user_id)
  values (target_user_id)
  on conflict (user_id) do nothing;

  return null;
end;
$$;

-- 3) claim_pool: var olan (sahipsiz) bir havuzu cagiran kullaniciya baglar.
create or replace function public.claim_pool(
  target_pool_id uuid,
  pool_name text default null,
  pool_location text default null
)
returns uuid
language plpgsql
security definer
set search_path = public
as $$
declare
  existing_owner uuid;
  next_sort integer;
begin
  if auth.uid() is null then
    raise exception 'Authentication required';
  end if;

  select user_id into existing_owner
  from public.pools
  where id = target_pool_id;

  if not found then
    raise exception 'pool not found';
```

```
  end if;

  if existing_owner is not null and existing_owner <> auth.uid() then
    raise exception 'pool already claimed';
  end if;

  select coalesce(max(sort_order), 0) + 1
  into next_sort
  from public.pools
  where user_id = auth.uid();

  update public.pools
  set
    user_id = auth.uid(),
    sort_order = case when existing_owner is null then next_sort else sort_order end,
    name = coalesce(nullif(btrim(pool_name), ''), name),
    location = coalesce(nullif(btrim(pool_location), ''), location)
  where id = target_pool_id;

  insert into public.pool_system_status (pool_id)
  values (target_pool_id)
  on conflict (pool_id) do nothing;

  return target_pool_id;
end;
$$;

revoke all on function public.claim_pool(uuid, text, text) from public;
grant execute on function public.claim_pool(uuid, text, text) to authenticated;
```

20260606020000_pool_status_last_sync.sql

```
-- ESP her PATCH'te last_sync gondermiyor (cihazda NTP saat yok).
-- Bu yuzden pool_system_status guncellendiginde last_sync sunucu saatiyle
-- otomatik tazelensin. Site "Son Veri" zamanini ve tazelik/missed-cycle
-- mantigini bu alandan okuyor.

create or replace function public.touch_pool_status()
returns trigger
language plpgsql
as $$
begin
  new.updated_at = timezone('utc', now());
  new.last_sync = timezone('utc', now());
  return new;
end;
$$;

drop trigger if exists set_pool_system_status_updated_at on
public.pool_system_status;
create trigger set_pool_system_status_updated_at
before update on public.pool_system_status
for each row
execute function public.touch_pool_status();
```

20260606030000_dev_mode_diagnostics.sql

```
-- Gelistirici modu ve ESP tanilama verisi.
-- dev_mode: kullanici tercihi; acikken sitede gelistirici bildirimleri gorunur.
-- diagnostics: ESP'nin gonderdigi ham tani verisi (voltajlar, heap, ADS durumu).

alter table public.user_preferences
  add column if not exists dev_mode boolean not null default false;

alter table public.pool_system_status
  add column if not exists diagnostics jsonb not null default '{}'::jsonb;
```

20260613000000_pool_alarm_settings.sql

```
create or replace function public.default_sensor_thresholds()
returns jsonb
language sql
immutable
as $$
  select jsonb_build_object(
    'ph', jsonb_build_object('warningMin', 6.5, 'warningMax', 8.5, 'criticalMin',
5.5, 'criticalMax', 9.5),
    'ec', jsonb_build_object('warningMin', 1.0, 'warningMax', 3.0, 'criticalMin',
0.5, 'criticalMax', 4.0),
    'tds', jsonb_build_object('warningMin', 150, 'warningMax', 1200, 'criticalMin',
50, 'criticalMax', 1800),
    'do', jsonb_build_object('warningMin', 5.0, 'warningMax', 12.0, 'criticalMin',
3.0, 'criticalMax', 15.0),
    'ldr', jsonb_build_object('warningMin', 20, 'warningMax', 90, 'criticalMin', 5,
'criticalMax', 98),
    'waterTemp', jsonb_build_object('warningMin', 20, 'warningMax', 30,
'criticalMin', 15, 'criticalMax', 35),
    'airTemp', jsonb_build_object('warningMin', 18, 'warningMax', 35, 'criticalMin',
10, 'criticalMax', 40),
    'humidity', jsonb_build_object('warningMin', 40, 'warningMax', 80, 'criticalMin',
20, 'criticalMax', 95)
  );
$$;

create or replace function public.default_alarm_settings()
returns jsonb
language sql
immutable
as $$
  select jsonb_build_object(
    'buzzerEnabled', true,
    'buzzerMuted', false,
    'alarmSensors', jsonb_build_object(
      'ph', true,
      'ec', true,
      'tds', true,
      'do', true,
      'ldr', true,
      'waterTemp', true,
      'airTemp', true,
      'humidity', true
    )
```

```
  );
$$;

alter table public.pools
  add column if not exists sensor_thresholds jsonb not null default
public.default_sensor_thresholds();

alter table public.pools
  add column if not exists alarm_settings jsonb not null default
public.default_alarm_settings();

update public.pools p
set sensor_thresholds = (
  select jsonb_object_agg(defaults.key, defaults.value ||
coalesce(p.sensor_thresholds -> defaults.key, '{}'::jsonb))
  from jsonb_each(public.default_sensor_thresholds()) as defaults(key, value)
);

update public.pools p
set alarm_settings =
  (public.default_alarm_settings() || coalesce(p.alarm_settings, '{}'::jsonb))
  || jsonb_build_object(
    'alarmSensors',
    (public.default_alarm_settings() -> 'alarmSensors') || coalesce(p.alarm_settings
-> 'alarmSensors', '{}'::jsonb)
  );

notify pgrst, 'reload schema';
```

20260613010000_sensor_event_logs.sql

```
create table if not exists public.sensor_event_logs (
  id bigint generated always as identity primary key,
  pool_id uuid not null references public.pools(id) on delete cascade,
  event_type text not null,
  sensor_id text,
  alarm_key text,
  severity text not null default 'info',
  value numeric(10, 2),
  threshold_min numeric(10, 2),
  threshold_max numeric(10, 2),
  message text not null default '',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default timezone('utc', now())
);

create index if not exists sensor_event_logs_pool_time_idx
  on public.sensor_event_logs (pool_id, created_at desc);

alter table public.sensor_event_logs enable row level security;

drop policy if exists "sensor_event_logs_select_own" on public.sensor_event_logs;
create policy "sensor_event_logs_select_own"
on public.sensor_event_logs
for select
to authenticated
using (
  exists (
    select 1
```

```
    from public.pools
    where pools.id = sensor_event_logs.pool_id
      and pools.user_id = auth.uid()
  )
);

drop policy if exists "sensor_event_logs_insert_own" on public.sensor_event_logs;
create policy "sensor_event_logs_insert_own"
on public.sensor_event_logs
for insert
to authenticated
with check (
  exists (
    select 1
    from public.pools
    where pools.id = sensor_event_logs.pool_id
      and pools.user_id = auth.uid()
  )
);

create or replace function public.default_alarm_settings()
returns jsonb
language sql
immutable
as $$
  select jsonb_build_object(
    'buzzerEnabled', true,
    'buzzerMuted', false,
    'mutedAlarmKey', null,
    'mutedAt', null,
    'alarmSensors', jsonb_build_object(
      'ph', true,
      'ec', true,
      'tds', true,
      'do', true,
      'ldr', true,
      'waterTemp', true,
      'airTemp', true,
      'humidity', true
    )
  );
$$;

alter table public.pools
  alter column alarm_settings set default public.default_alarm_settings();

update public.pools p
set alarm_settings =
  (public.default_alarm_settings() || coalesce(p.alarm_settings, '{}'::jsonb))
  || jsonb_build_object(
    'alarmSensors',
    (public.default_alarm_settings() -> 'alarmSensors') || coalesce(p.alarm_settings
-> 'alarmSensors', '{}'::jsonb)
  );

notify pgrst, 'reload schema';
```

20260615000000_device_refresh_interval_sync.sql

```
create or replace function public.default_alarm_settings()
returns jsonb
language sql
immutable
as $$
  select jsonb_build_object(
    'buzzerEnabled', true,
    'buzzerMuted', false,
    'readingIntervalMs', 3000,
    'mutedAlarmKey', null,
    'mutedAt', null,
    'alarmSensors', jsonb_build_object(
      'ph', true,
      'ec', true,
      'tds', true,
      'do', true,
      'ldr', true,
      'waterTemp', true,
      'airTemp', true,
      'humidity', true
    )
  );
$$;

alter table public.pools
  alter column alarm_settings set default public.default_alarm_settings();

update public.pools p
set alarm_settings =
  (public.default_alarm_settings() || coalesce(p.alarm_settings, '{}'::jsonb))
  || jsonb_build_object(
    'readingIntervalMs',
    coalesce(
      (
        select up.refresh_interval_ms
        from public.user_preferences up
        where up.user_id = p.user_id
      ),
      case
        when coalesce(p.alarm_settings ->> 'readingIntervalMs', '') ~ '^[0-9]+$'
          then (p.alarm_settings ->> 'readingIntervalMs')::integer
        else 3000
      end
    ),
    'alarmSensors',
    (public.default_alarm_settings() -> 'alarmSensors') || coalesce(p.alarm_settings
-> 'alarmSensors', '{}'::jsonb)
  );

notify pgrst, 'reload schema';
```